CASE STUDY 05 / SDE BACKEND

HR Roster Sync

A safe path from HR changes to employee transport

SUBMISSION	31 October 2026
STACK	Python 3.12 / FastAPI / SQLite / Docker Compose
REPOSITORY	github.com/arnavnaik22/HR-Roster-Sync
RUN	docker compose up --build

> **Central decision: a nightly file is a claim about the complete roster, not merely a list of rows to upsert. I stage it, measure what is absent, and hold a suspicious file before changing anyone's transport eligibility.**

The implementation and this report use synthetic identities only. The GitHub URL becomes public after the final push.

1. Problem and user impact

An employer's HR system sends near-real-time joiner, transfer, shift, and exit events to MoveInSync. It also sends a full nightly export of up to 200,000 employees. Duplicates, late delivery, invalid records, and cut-short files are normal failure modes. The transport roster must reflect legitimate HR changes without erasing fields set by the transport team.

The most consequential example is a truncated export. Suppose 200,000 people are active but the file ends after 120,000. A naive missing-row deletion could remove 80,000 employees from transport eligibility and cancel future rides. A correct sync must detect the large discrepancy and wait for a human decision. The case study specifies a 30% missing threshold, per-employee ordering, field ownership, an outbox, booking cancellation, a 200,000-row diff under 60 seconds, and tests for concurrency and failure.

This implementation makes that decision visible in a small operator workspace. The sample starts with 10 synthetic employees, uploads a file containing six, holds the file because four are missing, records an approval reason, then applies the change. A later-arriving old update is rejected. The workspace is an explanation of the real API flow; the sync rules are implemented in the backend.

2. Scope and source contract

Inputs. The event API accepts `CREATE`, `UPDATE`, and `DEACTIVATE` messages with a source version for each employee. The snapshot API accepts UTF-8 CSV and a source watermark in the request. The file contains employee ID, name, email, phone, manager, office, shift, and optional source version. Office codes must already exist for the tenant.

Essential assumption. The HR source assigns monotonically increasing versions for each employee. A nightly export is complete as of its watermark; every included row's version is at or below that watermark. If the sender cannot guarantee completeness, absence cannot safely mean departure. If it cannot provide comparable versions or a tie-breaker, this implementation cannot honestly guarantee order. These are integration contracts, not properties inferred from arrival time.

Output. An event returns `APPLIED`, `DUPLICATE`, or `STALE`, or a safe error code for invalid/conflicting input. A snapshot returns a run ID, row counts, missing count, and `READY` or `HELD`. A held run requires an operator to approve with a reason before application. Applied changes create outbox events. Deactivation cancels that employee's future confirmed bookings in the same database transaction.

3. Architecture

```
HR event / nightly CSV -> FastAPI boundary -> RosterStore -> SQLite
                            |                    |             |
                        auth + roles       sync rules    employees
                        tenant scope       validation    bookings
                        upload limit       ordering      snapshot staging
                                          safety brake   runs + errors
                                                         outbox -> relay -> consumer

Operator -> guided workspace -> same API -> held-run review + approval audit
```

FastAPI handles signed bearer tokens, role checks, tenant scope, bounded uploads, and safe HTTP responses. `RosterStore` owns validation, ordering, snapshot staging, reconciliation, booking effects, and outbox insertion. The database has employee and office tables keyed by tenant; a staging table keyed by run and employee; run and row-error tables; a tenant snapshot watermark; future bookings; and a pending outbox. Each event transition is one SQLite write transaction. Snapshot staging and snapshot application are separate transactions so an operator can inspect a held file without changing employees.

SQLite is a deliberate assessment choice: one command starts the app and the transaction boundary is easy to inspect. It is a single-writer demo database, not the proposed multi-node production store.

4. Ordering, idempotency, and ownership

The aggregate key is `(tenant_id, employee_id)`. On each event, the service validates the input and reads the stored source version. It then applies this rule:

Incoming state	Decision	Why
Version lower than stored	`STALE`	An old delivery cannot undo a newer state.
Same version, same canonical HR-owned state	`DUPLICATE`	Replaying the same source fact is a no-op.
Same version, different HR-owned state	`VERSION_CONFLICT`	The source must resolve conflicting claims.
Higher version	`APPLIED`	The source has supplied a newer state.

The canonical hash covers HR-owned fields, so equivalent `CREATE`, `UPDATE`, and snapshot `UPSERT` envelopes at the same version represent the same state. An explicit `CREATE` for an existing employee and `UPDATE` for a missing employee are rejected. The version and hash are stored with the employee. A database write lock and version recheck protect the local concurrent case; the concurrency test sends competing versions and confirms that state does not regress.

HR owns name, email, phone, manager, office, shift, and employment status. The transport admin owns pickup point and special assistance. HR SQL names only HR-owned columns; it does not replace a whole employee object. Deactivation preserves transport preferences for a possible reactivation. This is a concrete ownership boundary, not a last-writer-wins rule. The transport field endpoint requires an admin or operator role.

Example. A version 7 office transfer arrives before a version 6 shift change. Version 7 is applied. Version 6 returns `STALE`; the version 7 office and shift remain. Arrival order never becomes source order.

5. Nightly reconciliation and safety brake

The API spools uploads to disk after 1 MiB and rejects a body above 50 MiB. It reads CSV rows incrementally and stores valid rows in `snapshot_rows`; errors are saved with row number, a valid employee ID when available, and an error code. The report does not include phone, email, or pickup location. A bad row does not stop the remaining rows. Duplicate IDs within a file become row errors.

The database counts eligible active employees absent from staging with an indexed `NOT EXISTS` anti-join. Eligibility means the stored version is at or below the file watermark. The decision is:

```
missing_fraction = missing_eligible_active / eligible_active
HELD if eligible_active > 0 and missing_fraction >= threshold
default threshold = 0.30
```

Invalid rows stay absent from staging, so they can increase the missing fraction. This conservative behavior makes a damaged file more likely to pause. In the 10-person demo, four missing people produce 40%, so the run is `HELD` and none is deactivated. The operator must explicitly approve and provide a reason of at least eight characters. The run records operator identity and reason. A rejected or unapproved held file does not change the roster.

At application time the service obtains a write lock, rejects an old snapshot watermark, and rechecks per-employee versions. A live event newer than the file watermark remains untouched. Valid staged rows apply; a row that conflicts at application time is reported and skipped. Eligible missing employees are deactivated at the watermark. The employee changes, booking cancellations, outbox records, run status, and tenant watermark commit together. Reapplying an already applied run returns the previous result. Staging rows are removed after application.

The threshold is a safety brake, not proof that a file is complete: an incorrect export missing 20% could still pass. In production I would add expected-count checks, source checksums, historical-delta alerts, and source-side completion markers. Approval must be an informed decision, not a button pressed to clear a warning.

6. Downstream consistency and booking effects

For every applied employee change, the same transaction inserts an outbox row with a stable event ID. A `DEACTIVATE` also changes future `CONFIRMED` bookings to `CANCELLED` and emits a booking-cancellation event when there were bookings. The demo models cancellation as releasing the booking's claim on a seat; it does **not** maintain a separate vehicle seat-inventory table. A production booking service would own seat accounting and consume the cancellation idempotently.

The relay polls pending outbox rows and sends them to a configured HTTP consumer with the event ID as an idempotency key. Without a configured consumer, the local demo logs the event type and ID only. A database rollback leaves neither employee change nor event. If the database commits but the relay fails, the event remains pending for retry. The network guarantee is **at least once**, because a relay can crash after the consumer accepts an event but before the outbox row is marked sent. Consumers must deduplicate on the stable ID. I do not claim exactly-once delivery across the network.

7. Failure behavior and evidence

Failure	Behavior	Evidence in repository
Duplicate or late event	No state change	`test_replay_stale_and_conflict`
Competing event versions	State does not regress	`test_concurrent_versions_cannot_regress`
Invalid file row	Error recorded; other rows staged	`test_invalid_record_does_not_fail_snapshot`
Truncated export	Held before deactivation; reason required	`test_safety_brake_and_approval`, `test_csv_stream_hold_and_approval_audit`
Older snapshot after newer state	Newer state preserved	`test_older_snapshot_cannot_undo_newer_source_data`
Exception after employee write, before commit	Transaction rolled back, including outbox	`test_crash_rolls_back_employee_and_event`
Relay failure	Outbox remains pending and can retry	`api_demo.py` failure-injection walkthrough
Tenant or role mismatch	Access rejected	`test_auth_roles_and_tenant_boundary`
HR update to transport preference	Preference preserved	`test_transport_fields_survive_hr_update`

The failure test injects an exception before commit; it is not a literal operating-system process kill. The demo also injects a publisher failure to show retry. The tests establish these local behaviors, not multi-node fault tolerance.

8. Measured results and limits

The benchmark ran on Windows 11, Python 3.12.14, Intel64 Family 6 Model 186, with 12 logical CPUs. It preloaded 200,000 existing employees **before** timing. The timed operation staged a 200,000-row export with 1,000 missing and 1,000 new records and ran the missing-row comparison. A recorded run took **14.946 seconds** (about 13,381 rows per second), below the brief's 60-second target. Peak process working set was **24.0 MiB**; the SQLite database file and OS page cache are outside that number. The timed interval does **not** include applying 200,000 changes or remote delivery.

An in-process FastAPI benchmark of 1,000 event requests measured **59.1 requests/second**, **15.88 ms p50**, and **39.73 ms p99** on the same machine. It includes local SQLite commits but no network hop, external broker, or production load mix. These are reproducible local observations, not a production service-level objective. Run `python bench.py` and `python bench_api.py` to regenerate them.

CSV staging is O(n) time with bounded application memory; rows occupy O(n) database disk space. The indexed comparison is O(n + m) in the worst case over staged and stored employees. Each event uses indexed per-employee access. The first scaling constraint is SQLite's single writer, especially a large snapshot apply transaction. A growing outbox when the consumer is unavailable is another operational limit.

9. Security and operations

Tokens are signed and expire; claims carry tenant, role, and subject. The API derives tenant from the token, not a caller-supplied body field. HR can submit events and files; operators approve held runs; admin/operator can edit transport fields. Employee reads return only operator-facing fields. Errors use codes; structured request logs contain method, status, and elapsed time, not personal data. A local sample mode issues short-lived tokens for a fresh synthetic tenant and is enabled only in the localhost-bound Compose walkthrough. The demo signing secret must be replaced outside that environment.

Tenant-scoped metrics expose pending outbox count, oldest pending age, held runs, and invalid-row total. A small positive office-code cache has a five-second TTL and a 4,096-key LRU cap; an out-of-band office deletion could be accepted from cache for at most five seconds. Practical alerts would cover pending-event age, held files awaiting review, and sustained invalid-row growth. The implementation has simple retry and metrics, while exponential backoff, dead-letter handling, tracing, rate limits, and enterprise identity are production work.

10. Design trade-offs and next steps

The design favors correctness during source ambiguity: a suspicious file pauses, conflicting same-version events are rejected, and database failure blocks a write. The relay is asynchronous, so downstream consumers may see a short delay and must tolerate duplicates. The use of SQLite, a local token issuer, and one process keeps the assessment inspectable but limits concurrent throughput and deployment scale. Python is allowed by the brief; `RosterStore` encapsulates the core logic, while the API supplies the controller boundary. This is not a claim of Java-style inheritance or a full enterprise service architecture.

For a multi-instance production rollout, I would use PostgreSQL with guarded per-employee writes, a company identity provider with key rotation, a broker or CDC outbox relay, and idempotent booking consumers. I would avoid one very long write transaction for massive snapshot activation, add a source completion marker, and monitor outbox lag and approvals. The source version, field ownership, hold-before-apply, and event-ID rules should remain the same.

11. How to review and discuss it

1. Run `docker compose up --build` and open `http://localhost:8000/`. Start the synthetic workspace.

2. Show 10 active employees. Check the six-row file and show `HELD` with four missing; no one is deactivated yet.
3. Enter a meaningful reason, approve, and show six active employees plus the audit entry.
4. Send the out-of-order example from the screen; version 11 survives the later arrival of version 10.
5. Run `python -m unittest discover -s tests -v` (14 tests), `python api_demo.py` (API and retry), and `python bench.py` (200,000-row stage/diff) if asked for proof.

A defensible one-minute summary: “I treated a nightly file as a complete roster claim. I stage it and count missing active employees before applying changes. At 30% missing, the run pauses and requires an operator's recorded reason. Live events are ordered by source version, and HR changes cannot overwrite transport preferences. Employee changes and outbox rows commit together, so retries are safe. I tested concurrent versions and rollback, and timed the 200,000-row stage and diff. SQLite and the simple relay are demo choices; I would change those for a production rollout.”

12. Repository and references

- **Source and commit history:** https://github.com/arnavnaik22/HR-Roster-Sync (available after the final push).
- **Run instructions and full benchmark boundaries:** `README.md`.
- **Architecture, sequence, and failure table:** `DESIGN.md`.
- **Reproducible interview flow:** `DEMO_SCRIPT.md` and `INTERVIEW_GUIDE.md`.
- **Illustrated videos:** `assets/demo.mp4` and `assets/quick-tour.mp4`. These are illustrated walkthroughs, not a live screen recording.
- **Assignment source:** MoveInSync Backend Case Study 2026, Case Study 5: “HR Roster Sync - Idempotent, Out-of-Order, Self-Healing” (provided assessment PDF).
- **Product context:** MoveInSync HRMS integration guide.
- **Outbox pattern context:** Debezium Outbox Event Router documentation.
- **SQLite transaction behavior:** Python 3.12 sqlite3 documentation.

All sample identities are synthetic. The repository URL is configured locally but the user will perform the final public GitHub push.